Plan Offero — Parte 10 de 10

Cómo reportar tu trabajo: plantillas con ejemplos · Versión 2 · 29 de octubre de 2026

> Esta parte es para todo el equipo. Explica **qué reporte toca hacer en cada momento**, cómo llenarlo y cómo se ve uno bien hecho. Si dudas, copia el ejemplo verde y cambia los datos por los tuyos.

18. Tus reportes de un vistazo

Una semana normal de trabajo:

Momento	Qué haces	Reporte	Dónde
Cada día que trabajas	Cuentas en qué vas	**Daily** (18.1)	Discord #daily
Empiezas una tarea	La mueves a «En curso» en el tablero	— (ya existe el issue)	GitHub Projects
Terminas una tarea	Subes tu código para que lo revisen	**Pull request** (18.2)	GitHub
Encuentras algo que no funciona	Lo registras para que se arregle	**Reporte de bug** (18.3)	GitHub Issues
Te atascas más de 45 min	Pides ayuda con contexto	**Pedido de ayuda** (18.4)	Discord #ayuda
Domingo (cierre de sprint)	Review, Retro y Planning	**Agendas** (18.6)	Llamada en Discord
Terminas un módulo o un sprint	Actualizas el documento	**Actualización** (18.8)	docs/ del repositorio

El camino de una tarea, de principio a fin:

```
Backlog ──► Listo ──► En curso ──► En revisión ──► Hecho
                          │              │             │
                   la tomas y      abres el PR    el PR se fusionó
                   avisas en la    (18.2) y pides y está funcionando
                   Daily (18.1)    revisión       en producción
```

Reporte	Lo hacen	Frecuencia
Daily	Los 4	Cada día que trabajas
Pull request	Quien terminó una tarea	1 o más por semana
Bug	Quien encuentra el error (sobre todo en la prueba cruzada)	Cada vez que pase
Pedido de ayuda	Quien está atascado	Cuando hace falta (es bueno pedirla)
Historia de usuario	Diego (PO); otros pueden proponer	Al refinar el backlog
Registro de decisión	Diego o quien propone la decisión	Cuando se decide algo que afecta a todos
Actualización de la bóveda	Cada dueño de módulo; coordina Daniel	Al cerrar sprint o módulo

18.1 Daily: cuenta en qué vas

¿Cuándo lo uso?	Cada día que trabajes en el proyecto, aunque sea poco tiempo.
¿Quién lo hace?	Los cuatro. El facilitador de la semana además hace el resumen de las 10 p. m.
¿Dónde va?	Discord, canal #daily. Un solo mensaje por día.
¿Cuánto tarda?	3 minutos.

Paso a paso

1. Abre el tablero de GitHub Projects y mira tus tareas: así recuerdas sus IDs (T1-06, OFF-023…).
2. Copia la plantilla en #daily y llena las cuatro líneas.
3. Usa siempre el ID de la tarea. «Avancé en lo mío» no ayuda a nadie.
4. Si tienes un bloqueo, menciona (@) a quien te puede ayudar (Parte 3, 7.6).
5. Antes de las 10 p. m.

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
Daily · [día] [fecha] · [tu nombre]
Terminé: [ID y nombre de la tarea, o "nada aún"]
Sigo con: [ID y nombre de la tarea]
Bloqueos: [ninguno / qué te bloquea y @quién te puede ayudar]
¿Llego a mi fecha?: [sí / no, porque ... y propongo ...]
```

Ejemplo ya lleno — Wilson, Sprint 1

```
Daily · martes 10 nov · Wilson
Terminé: T1-11 SQL de requests (en pareja con Iker)
Sigo con: T1-12 Probar permisos de requests con 2 clientes y 1 proveedor
Bloqueos: el SELECT me devuelve vacío para el proveedor, @Iker ¿lo vemos mañana 15 min?
¿Llego a mi fecha?: sí, T1-12 queda el jueves
```

Errores comunes

✗ «Trabajé en el proyecto» sin decir qué tarea.

✗ Esperar a la Daily del día siguiente para contar un bloqueo que ya tienes.

✗ No escribir porque «no avancé nada»: igual se escribe, y se dice por qué.

✗ Escribir un párrafo largo: son 4 líneas.

18.2 Pull request: entrega tu trabajo para revisión

¿Cuándo lo uso?	Cuando terminaste una tarea (o un pedazo de menos de un día) y quieres que llegue a main.
¿Quién lo hace?	Quien hizo el código. Lo revisa al menos un compañero; si toca base de datos o login, Diego.
¿Dónde va?	GitHub → pestaña Pull requests → New pull request. La plantilla aparece sola (archivo .github/pull_request_template.md).
¿Cuánto tarda?	10 minutos.

Paso a paso

1. Sube tu rama: `git push`.
2. En GitHub, crea el PR desde tu rama hacia main.
3. Título: `[tipo](mN): qué hace`, el mismo estilo de los commits.
4. Llena la plantilla. En «Cómo probarlo» escribe los pasos como si el revisor nunca hubiera visto tu pantalla.
5. Marca cada casilla del checklist solo si es verdad. Si una no aplica, escribe «no aplica».
6. Pide la revisión en Discord #prs con el enlace.
7. Cuando el revisor comente, corrige con nuevos commits en la misma rama (no abras otro PR).

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
## Qué cambia
[Una o dos frases.]

Cierra #[número del issue] · [OFF-000] / [T0-00]

## Cómo probarlo
1. [Abre el Preview y entra como ...]
2. [Ve a ... y haz ...]
3. Permisos: [entra como otro rol y comprueba que ...]

## Capturas
[pega aquí las imágenes]

## Checklist
- [ ] Es menos de un día de trabajo
- [ ] Cumple los criterios de aceptación de la historia
- [ ] Validación en el servidor (no solo en el formulario)
- [ ] Permisos probados con dos cuentas de roles distintos
- [ ] Sin errores en la consola del navegador
- [ ] Si hay SQL: archivo en supabase/sql/, aplicado en dev, avisado en #prs
- [ ] Si hay tabla o vista nueva: revisó Iker (checklist de RLS)
- [ ] No hay claves, .env.local ni node_modules
- [ ] Puedo explicar cada parte de este código
- [ ] Actualicé el documento de contexto (o no hacía falta)
```

Ejemplo ya lleno — Daniel, Sprint 2

```
Título: feat(m2): agrega formulario Mi negocio

## Qué cambia
Agrega la página /negocios/mi-negocio para que un proveedor cree su negocio.

Cierra #31 · OFF-015

## Cómo probarlo
1. Abre el Preview y entra como daniel+proveedor1@gmail.com
2. Ve a "Mi negocio", llena todos los campos y guarda
3. Comprueba que aparece en /negocios
4. Permisos: entra como daniel+cliente1@gmail.com y abre
   /negocios/mi-negocio -> debe mandarte a la portada

## Capturas
(formulario vacío, formulario con error, negocio creado, vista de celular)

## Checklist
- [x] Es menos de un día de trabajo
- [x] Cumple los criterios de aceptación de la historia
- [x] Validación en el servidor (no solo en el formulario)
- [x] Permisos probados con dos cuentas de roles distintos
- [x] Sin errores en la consola del navegador
- [ ] Si hay SQL: no aplica
- [ ] Si hay tabla o vista nueva: no aplica
- [x] No hay claves, .env.local ni node_modules
- [x] Puedo explicar cada parte de este código
- [x] Actualicé el documento de contexto (o no hacía falta)
```

Errores comunes

- ✗ PR gigante con una semana de trabajo: nadie lo revisa bien y genera conflictos.
- ✗ «Cómo probarlo» vacío o con «pruébalo».
- ✗ Marcar casillas que no son ciertas.
- ✗ Abrir un PR nuevo para cada corrección: se corrige en el mismo.
- ✗ Fusionarlo tú mismo: lo fusiona Diego (merge commit).

18.3 Reporte de bug: algo no funciona

¿Cuándo lo uso?	Cuando algo no hace lo que dice su historia: en la prueba cruzada, probando lo tuyo o usando la app.
¿Quién lo hace?	Quien lo encuentra, aunque no sea su módulo.
¿Dónde va?	GitHub → Issues → New issue → «Reporte de bug». Luego se copia una línea a la sección 10 del documento de contexto.
¿Cuánto tarda?	5–10 minutos.

Paso a paso

1. Antes de reportar, intenta repetirlo una vez para estar seguro de los pasos.
2. Abre el navegador con F12 → pestaña Consola y copia el error en rojo, si hay.
3. Llena la plantilla. Lo más importante son los **pasos**: otra persona debe poder repetir el error siguiéndolos.
4. Gravedad: **bloquea** si impide el camino feliz (registrarse, publicar, cotizar, elegir); si no, **no bloquea**.
5. Asigna el issue al dueño del módulo y avísale en Discord si es grave.
6. Cuando el dueño lo corrija, quien lo reportó verifica y cierra el issue.

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
## Bug: [qué falla, en una línea]

Módulo: [M0-M7]          Historia: [OFF-000]
Gravedad: [Bloquea el camino feliz / No bloquea]
Dónde: [Local / Preview / Producción]   Navegador: [Chrome, celular ...]
Lo encontró: [nombre]    Fecha: [AAAA-MM-DD]

### Archivos afectados (si los sabes)
- [ruta del archivo]

### Mensaje de error
[pega aquí el error completo de la consola, la terminal o Vercel]

### Pasos para repetirlo
1. [Entra con la cuenta ... (rol: ...)]
2. [Ve a ...]
3. [Haz clic en ...]

### Resultado esperado
[lo que debería pasar según la historia]

### Resultado obtenido
[lo que pasó realmente] + captura
```

Ejemplo ya lleno — Daniel probando el módulo de Wilson

```
## Bug: se puede publicar una solicitud con fecha límite pasada

Módulo: M3          Historia: OFF-023
Gravedad: No bloquea
Dónde: Preview      Navegador: Chrome escritorio
Lo encontró: Daniel  Fecha: 2026-11-21

### Archivos afectados (si los sabes)
- src/app/solicitudes/nueva/page.js

### Mensaje de error
(no sale ningún error; esa es la falla)

### Pasos para repetirlo
1. Entra con daniel+cliente1@gmail.com (rol: cliente)
2. Ve a /solicitudes/nueva
3. Llena todo y pon fecha límite 01/11/2026
4. Clic en Publicar

### Resultado esperado
El servidor la rechaza y dice "La fecha límite debe ser futura".

### Resultado obtenido
Se publica y aparece como abierta. (captura adjunta)
```

Errores comunes

- ✗ «No sirve el login» sin pasos ni error.
- ✗ Reportar por WhatsApp y no crear el issue: se pierde.
- ✗ Tomarlo personal: encontrar bugs es bueno, para eso es la prueba cruzada.
- ✗ Arreglar en silencio el módulo de otro sin avisarle.

18.4 Pedido de ayuda: estás atascado

¿Cuándo lo uso?	Después de 45 minutos intentando solo (regla de atasco, Parte 3, 7.6).
¿Quién lo hace?	Quien está atascado. Responde quien sepa del tema (tabla de la Parte 3, 7.6).
¿Dónde va?	Discord, canal #ayuda.
¿Cuánto tarda?	5 minutos escribirlo. Si en 2 horas nadie responde, menciona a Diego.

Paso a paso

1. Escribe qué intentabas lograr, no solo el error.
2. Pega el error completo como texto (no solo una foto de la pantalla).
3. Di qué ya probaste: así nadie te sugiere lo mismo.
4. Menciona a la persona que más sabe del tema.
5. Cuando se resuelva, escribe en el mismo hilo cómo se arregló: le sirve al siguiente.

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
Ayuda · [tu nombre] · [tarea ID]
Qué intento: [lo que quieres lograr]
Archivo: [ruta]
Error: [pega el error completo]
Ya probé: [1. ... 2. ...]
@[persona que sabe del tema]
```

Ejemplo ya lleno — Iker, Sprint 1

```
Ayuda · Iker · T1-15
Qué intento: proteger /administrador para que solo entre el admin
Archivo: src/app/administrador/page.js
Error: me redirige a /login aunque entro con la cuenta de Diego (admin)
Ya probé: 1. cerrar y abrir sesión  2. revisar en Supabase que el rol es admin
@Diego
--- Solución (Diego): faltaba esperar (await) la función que trae el perfil.
```

Errores comunes

✗ Escribir solo «no me funciona».

✗ Quedarse atascado un día entero sin decir nada.

✗ Pedir ayuda y no contar después cómo se resolvió.

18.5 Historia de usuario: una funcionalidad nueva

¿Cuándo lo uso?	Cuando hay que agregar algo nuevo al backlog. Las 60 iniciales ya están en la Parte 5.
¿Quién lo hace?	Diego (PO). Cualquiera puede proponer una; Diego decide si entra y con qué prioridad.
¿Dónde va?	GitHub → Issues → New issue → «Historia de usuario» (archivo .github/ISSUE_TEMPLATE/historia.md).
¿Cuánto tarda?	10–15 minutos.

Paso a paso

1. Escribe la historia en la forma: **Como** [quién], **quiero** [qué], **para** [por qué].
2. Escribe los criterios con **Dado / Cuando / Entonces**. Siempre incluye uno de permisos («dado otro usuario..., entonces falla»).
3. Estima en puntos (1, 2, 3, 5 u 8). Si da más de 8, divídela en dos historias.
4. Diego le pone prioridad MoSCoW, responsable y sprint en el refinamiento.

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
## OFF-[000] · [título corto]

Épica: E[0-8]   Tipo: Funcional / Técnica / Seguridad / Producto
MoSCoW: Must / Should / Could   Puntos: [1, 2, 3, 5, 8]
Responsable: [nombre]   Sprint: [n]   Depende de: [OFF-000]

### Historia
Como [rol], quiero [acción], para [beneficio].

### Criterios de aceptación
- [ ] Dado [situación], cuando [acción], entonces [resultado].
- [ ] Permisos: dado [otro usuario o rol], cuando [intenta ...], entonces falla.

### Base de datos (si aplica)
Tabla: [...]  Campos: [...]  Políticas: [...]
```

Ejemplo ya lleno — propuesta hipotética de un cliente del piloto

```
## OFF-061 · Duplicar una solicitud anterior

Épica: E3   Tipo: Funcional
MoSCoW: Could   Puntos: 2
Responsable: Wilson   Sprint: 8   Depende de: OFF-026

### Historia
Como cliente, quiero duplicar una solicitud que ya hice,
para no volver a escribir todo cuando pido lo mismo.

### Criterios de aceptación
- [ ] Dado una solicitud mía, cuando pulso "Duplicar", entonces se abre
      el formulario lleno con sus datos y la fecha límite vacía.
- [ ] Permisos: dado la solicitud de otro cliente, cuando intento
      duplicarla, entonces falla.
```

Errores comunes

✗ Historias sin «para»: no se sabe por qué importa.

✗ Criterios vagos como «que funcione bien».

✗ Olvidar el criterio de permisos.

18.6 Agendas del domingo: Review, Retro y Planning

¿Cuándo lo uso?	Cada domingo de cierre de sprint, en un solo bloque de ~75 minutos (más largo en enero).
¿Quién lo hace?	Review y Planning: Diego. Retro: rota (nunca Diego). Todos participan.
¿Dónde va?	Llamada en Discord. La Review se hace sobre la URL de producción.
¿Cuánto tarda?	Review 20 min · pausa 5 · Retro 20 · Planning 30.

Qué tienes que llevar tú

Reunión	Tú llevas…
Review	Lo que terminaste (cumple la Definition of Done) abierto en producción, listo para mostrar en 2–3 minutos
Retro	Una o dos cosas que te ayudaron y una o dos que te frenaron esta semana
Planning	Tus horas disponibles la próxima semana (avisa si tienes parciales o viajes) y las tareas que crees que puedes tomar

Agenda para copiar en el evento de Discord

```
DOMINGO [fecha] · Cierre del Sprint [n] y Planning del Sprint [n+1]

REVIEW (20 min) · facilita Diego
00-02  Sprint Goal: [...]   ¿Se cumplió? Sí / Parcial / No
02-14  Cada uno muestra en producción lo terminado (2-3 min por persona)
14-17  Comentarios y preguntas
17-20  Velocidad: [ ] puntos terminados de [ ] comprometidos

--- pausa 5 min ---

RETRO (20 min) · facilita [nombre]   Formato: [ver Parte 8, 13.6]
00-02  ¿Cumplimos las acciones de la Retro pasada?
02-10  Cada uno escribe sus notas en silencio
10-15  Leer, agrupar y votar (2 votos cada uno)
15-20  Máximo 2 acciones: qué · quién · para cuándo

PLANNING (30 min) · facilita Diego
00-03  Horas disponibles de cada uno
03-08  Diego propone el Sprint Goal y las historias
08-22  Cada uno toma sus tareas y suma sus horas (sin pasarse)
22-27  ¿Quién espera algo de quién? Fechas
27-30  Sprint Goal final escrito en el milestone de GitHub
```

Ejemplo del resultado de una Retro — guardado en docs/retros.md

```
Retro Sprint 1 · dom 15 nov · facilitó Daniel · formato 4L
Me gustó:  la pareja Wilson-Iker para requests
Aprendí:   que una tabla sin políticas devuelve vacío sin dar error
Me faltó:  revisión rápida de PR (algunos esperaron 2 días)
Me habría gustado: saber antes qué categorías iban

| Acción                                      | Responsable | Para    |
|---------------------------------------------|-------------|---------|
| PR que no toquen BD los revisa otro, no Diego | Todos       | ya      |
| Lista de categorías publicada en docs/      | Diego       | mar 17  |
```

18.7 Registro de decisión

¿Cuándo lo uso?	Cuando se decide algo que afecta a más de una persona: una regla del producto, una tabla, una convención o un cambio de fechas.
¿Quién lo hace?	Diego, o quien propuso la decisión, una vez acordada.
¿Dónde va?	Sección 11 del documento de contexto, en docs/.
¿Cuánto tarda?	5 minutos.

Paso a paso

1. Usa el siguiente número libre (D-10, D-11…).
2. Escribe la decisión en una o dos frases claras.
3. Anota qué otras opciones hubo y por qué se eligió esta: así nadie reabre la discusión en un mes.
4. Si cambia algo del plan (una tabla, una fecha, una historia), dilo en «Consecuencias».

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
### D-[00] · [título]
- Fecha: [AAAA-MM-DD]
- Estado: Propuesta / Tomada / Reemplazada por D-[00]
- Decide: [Diego / el equipo / dueño del módulo]
- Contexto: [qué problema había]
- Decisión: [qué se decidió]
- Alternativas: [opción A ... / opción B ...]
- Motivo: [por qué esta]
- Consecuencias: [qué cambia]
```

Ejemplo ya lleno — ejemplo de cómo podría quedar la D-09 (la decisión real la toma Diego)

```
### D-09 · Correos de confirmación en el MVP
- Fecha: 2026-10-30
- Estado: Tomada
- Decide: Diego
- Contexto: el correo incluido en Supabase no llega a usuarios reales
- Decisión: desactivar "Confirm email" en offero-prod durante el MVP
- Alternativas: SMTP propio (agrega un proveedor externo)
- Motivo: las empresas del piloto deben poder registrarse ya
- Consecuencias: "recuperar contraseña" queda fuera del MVP
```

Errores comunes

✗ Decidir en un chat de WhatsApp y no registrarlo.

✗ No escribir las alternativas: en un mes nadie recuerda por qué.

18.8 Actualización del documento de contexto

¿Cuándo lo uso?	Al cerrar cada sprint y cada vez que un módulo se termina.
¿Quién lo hace?	Cada dueño completa su módulo; Daniel coordina y revisa que esté todo.
¿Dónde va?	El documento de contexto en docs/, mediante un PR de tipo docs.
¿Cuánto tarda?	10–15 minutos por persona.

Paso a paso

1. Sección 8: cambia el estado de tu módulo (No iniciado / En curso / Terminado) y si está publicado.
2. Sección 9: anota limitaciones conocidas que siguen abiertas.
3. Sección 10: copia los bugs del sprint (una línea cada uno, con su estado).
4. Sección 11: lista las decisiones nuevas (el detalle va en su registro).
5. Sección 3: solo si cambió algo del equipo.

Plantilla en blanco (copia y reemplaza lo que está entre corchetes)

```
## Actualización · Sprint [n] · [fecha]

### Sección 8 · Estado de los módulos
| Módulo | Responsable | Estado | Publicado | Notas |
|--------|-------------|--------|-----------|-------|
| M[ ]   | [nombre]    | [...]  | Sí / No   | [...] |

### Sección 9 · Problemas conocidos
- [...]

### Sección 10 · Errores
| Fecha | Módulo | Error | Estado |
|-------|--------|-------|--------|
| [...] | [...]  | [...] | Abierto / Corregido |

### Sección 11 · Decisiones nuevas
- D-[00] · [título]
```

Ejemplo ya lleno — cierre del Sprint 2

```
## Actualización · Sprint 2 · dom 22 nov

### Sección 8 · Estado de los módulos
| Módulo | Responsable | Estado    | Publicado | Notas                     |
|--------|-------------|-----------|-----------|---------------------------|
| M0     | Daniel      | Terminado | Sí        |                           |
| M1     | Diego       | Terminado | Sí        | Falta login con Google    |
| M2     | Daniel      | En curso  | Parcial   | Falta listado y detalle   |
| M3     | Wilson      | En curso  | Parcial   | Formulario listo          |

### Sección 10 · Errores
| Fecha  | Módulo | Error                          | Estado    |
|--------|--------|--------------------------------|-----------|
| 21 nov | M3     | Acepta fecha límite pasada     | Corregido |
```

Errores comunes

✗ Dejar que se acumule varios sprints: después nadie recuerda qué pasó.

✗ Marcar Terminado un módulo que no cumple la Definition of Done (Parte 3, 7.2).

19. Próximos pasos inmediatos

Las 10 primeras acciones, en orden (desde el jueves 29 de octubre).

#	Acción	Responsable	Plazo
1	Avisar por WhatsApp que acepten las invitaciones de GitHub y de Supabase	Diego	**Hoy**
2	Instalar Node.js LTS y Git y configurar Git (Parte 7, día 29)	Iker, Daniel, Wilson	**Hoy** o mañana
3	Regenerar las guías y la Parte 1 con las fechas de la Parte 4 y enviar a cada uno su guía	Diego	**Hoy**
4	Configurar Auth en offero-dev y offero-prod y decidir D-09 (correos)	Diego	**Mañana** (vie 30)
5	Crear el proyecto Next.js con Tailwind y abrir el PR del proyecto base (M0)	Daniel, con Wilson	vie 30 – sáb 31
6	Hablar con Iker sobre su rol: M7, M6 y revisor de seguridad	Diego	**Mañana**
7	Confirmar con el profesor la entrega del vie 29 ene y las Reviews de hitos	Diego	Esta semana
8	Primer despliegue en Vercel (Production → prod, Preview → dev)	Diego y Daniel	dom 1 nov
9	Preguntar al equipo por finales y disponibilidad en enero; a Wilson, si su app usaba SQL	Diego	Esta semana
10	Lista inicial de categorías reales y agendar las 5 entrevistas (guion en la Parte 9)	Diego	Antes del lun 9 nov

> **Para recordar:** la Daily cada día, un PR por tarea terminada, un issue por cada bug, pedir ayuda a los 45 minutos y el documento al día al cerrar cada sprint. Si haces esas cinco cosas, el equipo siempre sabe en qué va el proyecto.